#DL-01

```
from sklearn.datasets import load_boston
import pandas as pd

boston_dataset = load_boston()

df = pd.DataFrame(boston_dataset.data, columns=boston_dataset.feature_names)
df['MEDV'] = boston_dataset.target

df.head(n=10)

df = pd.read_csv("./1_boston_housing.csv")

from sklearn.model_selection import train_test_split

X = df.loc[:, df.columns != 'MEDV']
y = df.loc[:, df.columns == 'MEDV']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=123)

from sklearn.preprocessing import MinMaxScaler
mms = MinMaxScaler()
mms.fit(X_train)
X_train = mms.transform(X_train)
X_test = mms.transform(X_test)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()

model.add(Dense(128, input_shape=(13, ), activation='relu', name='dense_1'))
model.add(Dense(64, activation='relu', name='dense_2'))
model.add(Dense(1, activation='linear', name='dense_output'))

model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.summary()

Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 dense_1 (Dense)             (None, 128)               1792

 dense_2 (Dense)             (None, 64)                8256

 dense_output (Dense)        (None, 1)                 65
=================================================================
```

```
Total params: 10,113
Trainable params: 10,113
Non-trainable params: 0
_________________________________________________________________

history = model.fit(X_train, y_train, epochs=100, validation_split=0.05, verbose = 1)

Epoch 1/10
11/11 [==============================] - 2s 57ms/step - loss: 1468.4281 - mae:
32.0408 - val_loss: 104.8761 - val_mae: 6.1343
Epoch 2/10
11/11 [==============================] - 0s 9ms/step - loss: 291.9155 - mae:
13.9797 - val_loss: 149.5561 - val_mae: 11.4423
Epoch 3/10
11/11 [==============================] - 0s 7ms/step - loss: 151.9729 - mae:
10.3968 - val_loss: 141.8171 - val_mae: 7.9483
Epoch 4/10
11/11 [==============================] - 0s 8ms/step - loss: 87.9686 - mae: 7.0836 -
val_loss: 96.4421 - val_mae: 8.3894
Epoch 5/10
11/11 [==============================] - 0s 10ms/step - loss: 66.2962 - mae: 5.8493
- val_loss: 79.1745 - val_mae: 5.6716
Epoch 6/10
11/11 [==============================] - 0s 13ms/step - loss: 59.8653 - mae: 5.6803
- val_loss: 81.4188 - val_mae: 5.8136
Epoch 7/10
11/11 [==============================] - 0s 7ms/step - loss: 57.8316 - mae: 5.3080 -
val_loss: 78.1150 - val_mae: 6.1632
Epoch 8/10
11/11 [==============================] - 0s 8ms/step - loss: 55.0569 - mae: 5.0945 -
val_loss: 78.3047 - val_mae: 6.4755
Epoch 9/10
11/11 [==============================] - 0s 4ms/step - loss: 56.2966 - mae: 5.2586 -
val_loss: 78.7134 - val_mae: 6.3377
Epoch 10/10
11/11 [==============================] - 0s 4ms/step - loss: 56.3019 - mae: 5.3718 -
val_loss: 85.3209 - val_mae: 7.5740
mse_nn, mae_nn = model.evaluate(X_test, y_test)

print('Mean squared error on test data: ', mse_nn)
print('Mean absolute error on test data: ', mae_nn)

Mean squared error on test data:  25.959447860717773

Mean absolute error on test data:  3.8496510982513428
```

#DL-02

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import numpy as np

# Load IMDB dataset (Top 10,000 most frequent words)
num_words = 10000
(x_train, y_train), (x_test, y_test) =
keras.datasets.imdb.load_data(num_words=num_words)

# Pad sequences so all reviews have same length
max_length = 200

x_train = keras.preprocessing.sequence.pad_sequences(x_train,
                                    maxlen=max_length)

x_test = keras.preprocessing.sequence.pad_sequences(x_test,
                                   maxlen=max_length)

# Build Deep Neural Network model
model = keras.Sequential([
   layers.Embedding(input_dim=num_words,
               output_dim=128,
               input_length=max_length),

   layers.Flatten(),

   layers.Dense(64, activation='relu'),
   layers.Dense(32, activation='relu'),

   layers.Dense(1, activation='sigmoid')  # Binary classification
])

# Compile model
model.compile(optimizer='adam',
           loss='binary_crossentropy',
           metrics=['accuracy'])

# Train model
history = model.fit(x_train,
                y_train,
                epochs=5,
                batch_size=512,
                validation_split=0.2)

# Evaluate model
```

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)

print("\nTest Accuracy:", test_accuracy)


#OUTPUT

Epoch 1/5
40/40 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5s 95ms/step - accuracy: 0.7215 - loss: 0.5412 - val_accuracy: 0.8501 - val_loss: 0.3567

Epoch 2/5
40/40 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3s 73ms/step - accuracy: 0.9104 - loss: 0.2453 - val_accuracy: 0.8756 - val_loss: 0.3102

Epoch 3/5
40/40 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3s 72ms/step - accuracy: 0.9627 - loss: 0.1405 - val_accuracy: 0.8809 - val_loss: 0.3304

Epoch 4/5
40/40 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3s 74ms/step - accuracy: 0.9856 - loss: 0.0723 - val_accuracy: 0.8798 - val_loss: 0.3721

Epoch 5/5
40/40 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3s 72ms/step - accuracy: 0.9941 - loss: 0.0306 - val_accuracy: 0.8754 - val_loss: 0.4518

782/782 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8723 - loss: 0.4627

Test Accuracy: 0.8723
```

#DL-03

```
from tensorflow.keras.datasets import fashion_mnist

(train_x, train_y), (test_x, test_y) = fashion_mnist.load_data()


from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, MaxPooling2D, Conv2D

model = Sequential()

model.add(Conv2D(filters=64,kernel_size=(3,3),activation='relu',input_shape=(28, 28, 1)))

# Adding maxpooling layer to get max value within a matrix
model.add(MaxPooling2D(pool_size=(2,2)))

model.add(Flatten())
model.add(Dense(128, activation = "relu"))
model.add(Dense(10, activation = "softmax"))

model.summary()

Model: "sequential_2"
_________________________________________________________________
 Layer (type)            Output Shape           Param #
=================================================================
 flatten_1 (Flatten)     (None, 784)            0

 dense_8 (Dense)         (None, 128)            100480

 dense_9 (Dense)         (None, 10)             1290

=================================================================
Total params: 101,770
Trainable params: 101,770
Non-trainable params: 0
_________________________________________________________________

model.compile(optimizer = 'adam', loss = 'sparse_categorical_crossentropy', metrics =
['accuracy'])

model.fit(train_x.astype(np.float32), train_y.astype(np.float32), epochs = 5,
validation_split = 0.2)

Epoch 1/5
1500/1500 [==============================] - 7s 5ms/step - loss: 0.4881 - accuracy:
0.8302 - val_loss: 0.5287 - val_accuracy: 0.8273
```

```
Epoch 2/5
1500/1500 [==============================] - 6s 4ms/step - loss: 0.4893 - accuracy:
0.8298 - val_loss: 0.5376 - val_accuracy: 0.8243
Epoch 3/5
1500/1500 [==============================] - 7s 5ms/step - loss: 0.4774 - accuracy:
0.8342 - val_loss: 0.5451 - val_accuracy: 0.8282
Epoch 4/5
1500/1500 [==============================] - 6s 4ms/step - loss: 0.4751 - accuracy:
0.8361 - val_loss: 0.5717 - val_accuracy: 0.8299
Epoch 5/5
1500/1500 [==============================] - 7s 5ms/step - loss: 0.4753 - accuracy:
0.8363 - val_loss: 0.5278 - val_accuracy: 0.8255

<keras.callbacks.History at 0x7fbcee0aceb0>

loss, acc = model.evaluate(test_x, test_y)

313/313 [==============================] - 1s 2ms/step - loss: 0.5724 - accuracy:
0.8171

labels = ['t_shirt', 'trouser', 'pullover', 'dress', 'coat', 'sandal', 'shirt', 'sneaker',   'bag',
'ankle_boots']

predictions = model.predict(test_x[:1])

1/1 [==============================] - 0s 73ms/step

import numpy as np

label = labels[np.argmax(predictions)]

import matplotlib.pyplot as plt
print(label)
plt.imshow(test_x[:1][0])
plt.show

ankle_boots

<function matplotlib.pyplot.show(close=None, block=None)>

[]
```

#DL-04

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, SimpleRNN

# Load dataset
dataset_train = pd.read_csv('Google_Stock_Price_Train.csv')
training_set = dataset_train.iloc[:, 1:2].values

# Feature scaling
sc = MinMaxScaler(feature_range=(0,1))
training_set_scaled = sc.fit_transform(training_set)

# Create dataset with 60 time steps
X_train = []
y_train = []

for i in range(60, len(training_set_scaled)):
   X_train.append(training_set_scaled[i-60:i, 0])
   y_train.append(training_set_scaled[i, 0])

X_train, y_train = np.array(X_train), np.array(y_train)

# Reshape dataset for RNN input
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))

# Build RNN model
model = Sequential()

model.add(SimpleRNN(units=50,
                activation='tanh',
                return_sequences=True,
                input_shape=(X_train.shape[1],1)))

model.add(SimpleRNN(units=50))

model.add(Dense(units=1))

# Compile model
model.compile(optimizer='adam',
          loss='mean_squared_error')

# Train model
```

```
model.fit(X_train, y_train,
        epochs=20,
        batch_size=32)

# Load test dataset
dataset_test = pd.read_csv('Google_Stock_Price_Test.csv')
real_stock_price = dataset_test.iloc[:,1:2].values

# Prepare test data
dataset_total = pd.concat((dataset_train['Open'],
                    dataset_test['Open']), axis=0)

inputs = dataset_total[len(dataset_total)
                - len(dataset_test) - 60:].values

inputs = inputs.reshape(-1,1)
inputs = sc.transform(inputs)

X_test = []

for i in range(60, 80):
   X_test.append(inputs[i-60:i, 0])

X_test = np.array(X_test)

X_test = np.reshape(X_test,
                (X_test.shape[0],
                 X_test.shape[1],
                 1))

# Predict stock prices
predicted_stock_price = model.predict(X_test)

predicted_stock_price = sc.inverse_transform(predicted_stock_price)

# Plot results
plt.plot(real_stock_price, color='red',
       label='Real Google Stock Price')

plt.plot(predicted_stock_price,
       color='blue',
       label='Predicted Google Stock Price')

plt.title('Google Stock Price Prediction using RNN')
plt.xlabel('Time')
plt.ylabel('Stock Price')
plt.legend()
```

```
plt.show()

#OUTPUT
Epoch 1/20
38/38 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3s - loss: 0.0345
Epoch 2/20
38/38 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s - loss: 0.0059
Epoch 3/20
38/38 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s - loss: 0.0047
...
Epoch 20/20
38/38 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s - loss: 0.0018
```